```
import re
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.decomposition import PCA

tf.random.set_seed(42)

text = """deep learning is a part of machine learning. machine
learning uses data to train models.
neural networks are models used in deep learning. a neural network has
layers of neurons.
the model learns patterns from data. natural language processing uses
deep learning models to understand text.
word embeddings represent words as vectors. the cbow model predicts a
target word from its context words.
skip gram predicts context words from a target word. embeddings
capture the meaning of words in vectors."""
# EXAM DAY: replace with any paragraph you like (a longer one gives
better embeddings)

words = re.findall(r"[a-z]+", text.lower())
vocab = sorted(set(words))
word2idx = {w: i for i, w in enumerate(vocab)}
idx2word = {i: w for w, i in word2idx.items()}
vocab_size = len(vocab)
print("Words:", len(words), "| Vocabulary:", vocab_size)
```

```
Words: 80 | Vocabulary: 46
```

```
WINDOW = 2        # words taken on each side of the target

X, y = [], []
for i in range(WINDOW, len(words) - WINDOW):
    context = words[i - WINDOW:i] + words[i + 1:i + WINDOW + 1]     #
2 words before + 2 words after
    X.append([word2idx[w] for w in context])
    y.append(word2idx[words[i]])
X, y = np.array(X), np.array(y)

print(X.shape, y.shape)                                    # (pairs, 4)
and (pairs,)
print([idx2word[i] for i in X[0]], "->", idx2word[y[0]])  # one
example pair
```

```
(76, 4) (76,)
['deep', 'learning', 'a', 'part'] -> is
```

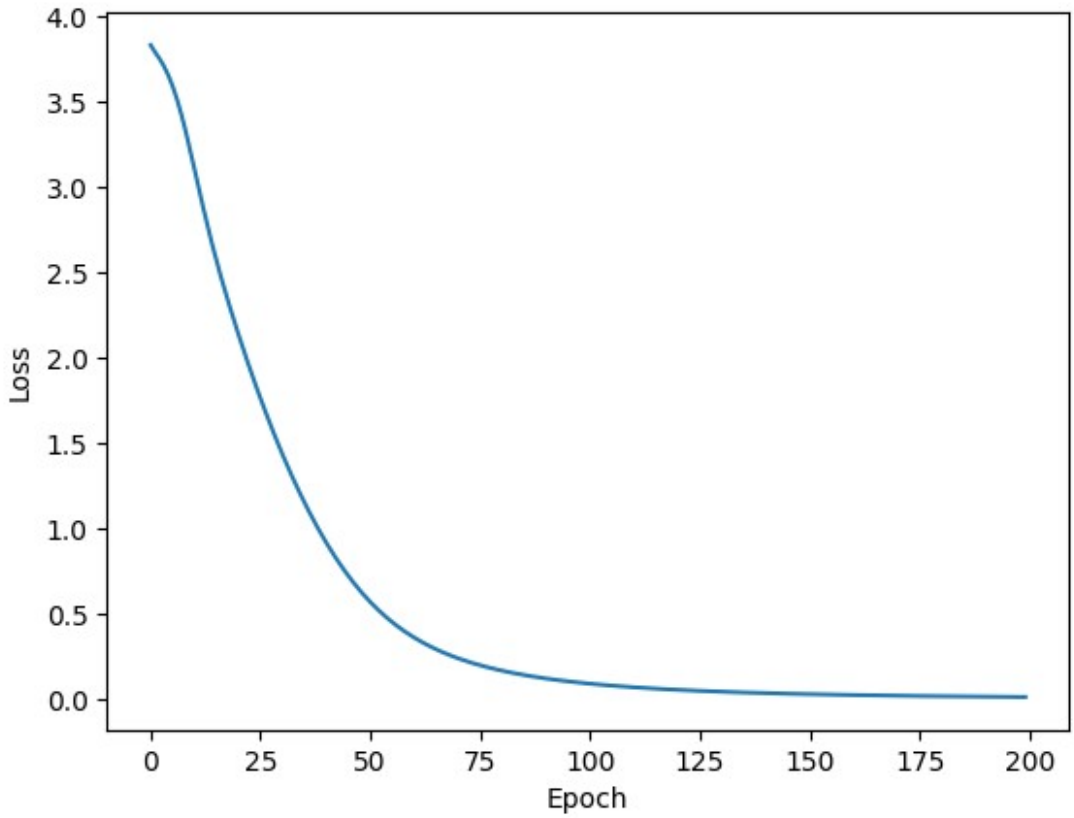
```
EMB_DIM = 10

model = keras.Sequential([
    layers.Input(shape=(2 * WINDOW,)),
    layers.Embedding(vocab_size, EMB_DIM),       # lookup table: one
vector per word
    layers.GlobalAveragePooling1D(),             # average the 4
context vectors
    layers.Dense(vocab_size, activation="softmax"),   # probability of
each word being the target
])

model.compile(loss="sparse_categorical_crossentropy",
optimizer=keras.optimizers.Adam(0.01), metrics=["accuracy"])
history = model.fit(X, y, epochs=200, batch_size=16, verbose=0)

print("Final loss:", history.history["loss"][-1], "| accuracy:",
history.history["accuracy"][-1])
plt.plot(history.history["loss"])
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.show()
```

```
Final loss: 0.01703515648841858 | accuracy: 1.0
```



```
def predict_word(context_words):
    idx = np.array([[word2idx[w] for w in context_words]])
    return idx2word[int(model.predict(idx, verbose=0).argmax())]

print(predict_word(["cbow", "model", "a", "target"]))     # the true
middle word here is "predicts"

predicts

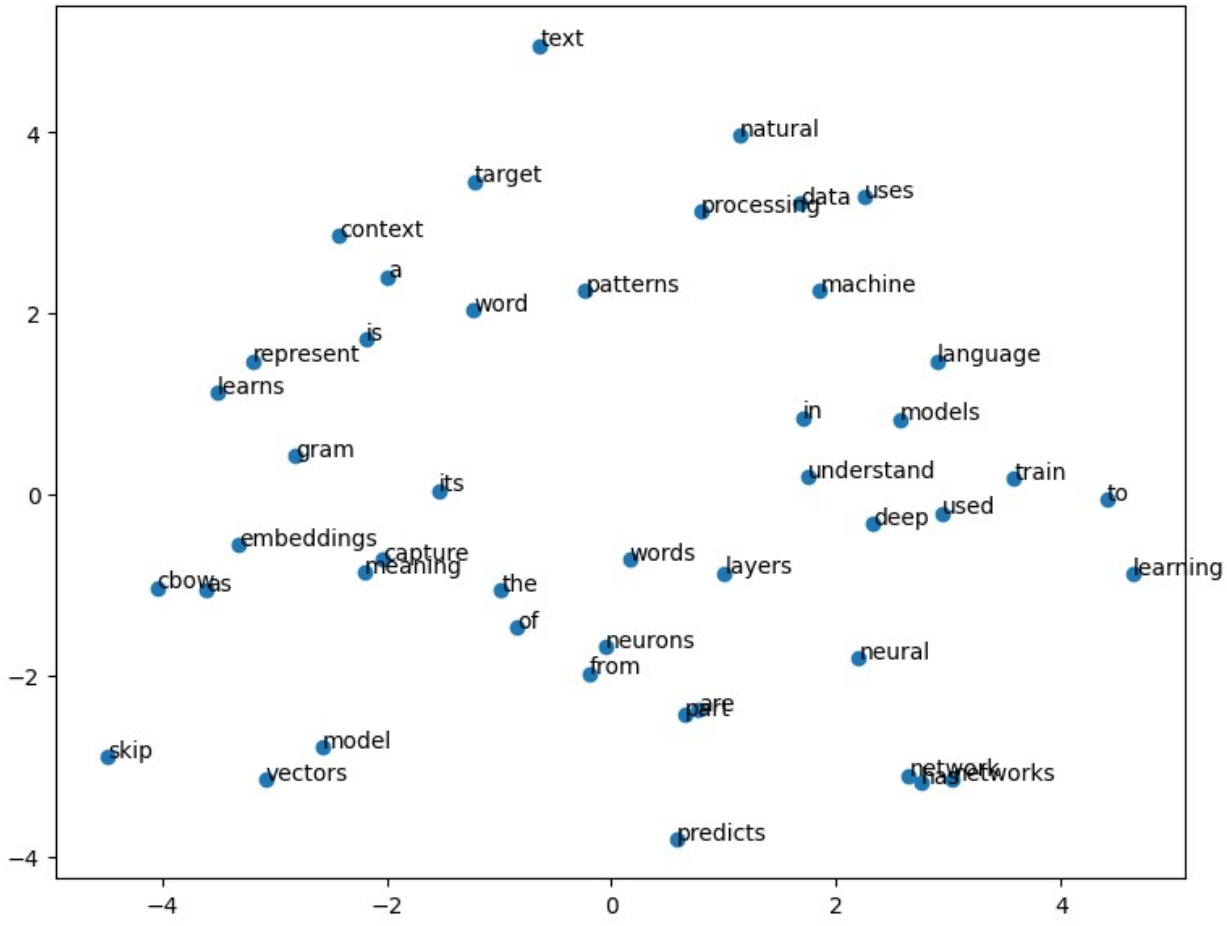
emb = model.layers[0].get_weights()[0]      # embedding matrix, shape
(vocab_size, EMB_DIM)

def similar(word, k=3):
    v = emb[word2idx[word]]
    sims = emb @ v / (np.linalg.norm(emb, axis=1) * np.linalg.norm(v))
# cosine similarity
    top = np.argsort(-sims)[1:k + 1]
# skip the word itself
    return [(idx2word[i], round(float(sims[i]), 2)) for i in top]

print("Similar to 'model':   ", similar("model"))
print("Similar to 'learning':", similar("learning"))

pts = PCA(n_components=2).fit_transform(emb)         # squash 10
dimensions to 2 for plotting
plt.figure(figsize=(9, 7))
plt.scatter(pts[:, 0], pts[:, 1])
for i in range(vocab_size):
    plt.annotate(idx2word[i], pts[i])
plt.show()

Similar to 'model':    [('skip', 0.82), ('as', 0.55), ('cbow', 0.53)]
Similar to 'learning': [('part', 0.57), ('network', 0.51), ('to',
0.46)]
```


text
natural
target
processing
data
uses
context
a
patterns
machine
word
is
represent
language
learns
in
models
gram
understand
train
its
to
deep
used
embeddings
capture
words
layers
learning
meaning
cbow
as
the
of
neurons
neural
from
are
skip
model
vectors
network
networks
predicts
4
2
0
−2
−4
−4
−2
0
2
4